

CS525 Spring 2026 Final Project Report

LAMBDA $\rightarrow$ JavaScript compiler
with type inference and closure conversion

Youngsun Lim

1. Introduction to Church’s lambda-calculus (Item 2a)

The lambda-calculus, introduced by Alonzo Church, is a minimal formal language of computation. Its grammar contains only three forms:

```
e ::= x          -- variable
    |  $\lambda x. e$     -- abstraction (function)
    | e1 e2       -- application
```

Despite this austerity, the calculus is **Turing-complete**: numbers (Church numerals), booleans, pairs, conditionals, and recursion (via the Y combinator) can all be encoded purely with abstraction and application.

The two operational rules are:

- **β -reduction** — $(\lambda x. e) v \rightarrow e[x := v]$. Substitute the argument for the parameter inside the body.
- **α -conversion** — $\lambda x. e \equiv \lambda y. e[x := y]$ (when y is fresh). Names of bound variables don’t matter.

Lambda-calculus matters in compiler construction because it is the **theoretical core** of every functional programming language. Once a compiler can handle lambda + application correctly — including the treatment of free variables across nested abstractions — extending it with primitives, conditionals, tuples, lists, and so on is largely mechanical. The hard part is precisely what closure conversion solves: *how does a nested function remember the values of variables that were in scope at the time of its creation?*

2. The LAMBDA programming language (Item 2b)

LAMBDA is the source language for our compiler. It is an explicitly-typed (with optional inference of existential variables) extended lambda-calculus. The two ASTs are:

2.1 Types — styp

Constructor	Meaning
STbas(name)	base type (int, bool, string)
STxyz(ref opt)	existential / metavariable used during inference
STtup(T1, T2)	binary product type $T_1 * T_2$
STfun(T1, T2)	function type $T_1 \rightarrow T_2$
STlazy(T)	suspended computation of type T
STlist(T)	list of T s
STarry(T)	mutable array of T
STstcn(T)	(lazy) stream constructor over T

STxyz is the workhorse of inference: each occurrence is a fresh “type unknown” with a mutable cell. As inference proceeds, unification fills the cell with a concrete type.

2.2 Terms — dexp

Each constructor with its **typing rule** and **operational meaning**:

Term	Type rule (sketch)	Meaning
DEint(n)	$: \text{int}$	integer literal
DEbtf(b)	$: \text{bool}$	boolean literal
DEstr(s)	$: \text{string}$	string literal
DEvar(x)	$\Gamma(x) = T \Rightarrow : T$	variable lookup
DElam(x, e)	$\Gamma, x:\alpha \vdash e : \beta \Rightarrow : \alpha \rightarrow \beta$ (fresh α)	anonymous function
DElam1(x, T, e)	$\Gamma, x:T \vdash e : \beta \Rightarrow : T \rightarrow \beta$	anonymous function (annotated arg)
DEapp(f, a)	$f : \alpha \rightarrow \beta, a : \alpha \Rightarrow : \beta$	function application
DEopr(p, args)	per-operator rule	primitive (+, <=, list_head, ...)
DEfst(e) / DEsnd(e)	$e : \alpha * \beta \Rightarrow : \alpha / : \beta$	tuple projection
DEtup(e1, e2)	$: T_1 * T_2$	tuple construction
DEif0(t, c, a)	$t:\text{bool}, c:T, a:T \Rightarrow : T$	conditional
DEFix(f, x, e)	$f:\alpha \rightarrow \beta, x:\alpha \vdash e:\beta \Rightarrow : \alpha \rightarrow \beta$	recursive function
DEFix1(f, x, T1, e, T2)	annotated version	recursive fn with type annotations
DElet(x, d, s)	$d:T, \Gamma, x:T \vdash s:U \Rightarrow : U$	local binding
DElist_nil()	$: \text{list } \alpha$	empty list
DElist_cons(h, t)	$h:T, t:\text{list } T \Rightarrow : \text{list } T$	list cons
DElazy(e)	$e:T \Rightarrow : \text{lazy } T$	suspension
DEstcn_nil()	$: \text{stcn } \alpha$	empty stream
DEstcn_cons(h, t)	$h:T, t:\text{lazy}(\text{stcn } T) \Rightarrow : \text{stcn } T$	stream cons (tail is lazy)
DEarray_size\$val(n, v)	$n:\text{int}, v:T \Rightarrow : \text{array } T$	array of n copies of v
DEarray_size\$fun(n, f)	$n:\text{int}, f:\text{int} \rightarrow T \Rightarrow : \text{array } T$	array $[f(0), f(1), \dots, f(n-1)]$
DEanno(e, T)	$e:T \Rightarrow : T$	explicit type ascription

2.3 Operational semantics

LAMBDA evaluates eagerly (call-by-value), with two exceptions: **DElazy** constructs a memoised thunk evaluated only when forced via the **lazy_force** primitive, and **DEstcn_cons** keeps its tail suspended, giving rise to potentially-infinite streams.

3. Type-checking (Item 2c)

The type-checker is a textbook implementation of **Hindley–Milner-style inference using unification**, but without let-polymorphism (each **STxyz** is a single existential, not a generalised universal).

3.1 The four pillars

- **styp_new()** — create a fresh existential metavariable.
- **prune(t)** — follow the chain of **STxyz** $\rightarrow \dots$ references until a non-**STxyz** (or unsolved **STxyz**) is reached. Implements path compression as a side effect.
- **occurs(α , T)** — does α appear inside T ? Required to prevent the construction of infinite types (e.g. $\alpha = \text{list } \alpha$).
- **unify(T1, T2)** — make two types equal, by either resolving an unsolved **STxyz** or recursing structurally. Raises **TypeError** on a mismatch.

3.2 Inference algorithm

`dexp_tinfer(de) = prune(dexp_tinfer1(de,  $\emptyset$ )).`

`dexp_tinfer1(de, ctx)` walks the AST. For each constructor it:

1. allocates fresh metavariables for any “unknown” types (parameter types, list element types, etc.);
2. recursively infers the types of subterms in an extended context;
3. calls `unify` to enforce the typing rule’s constraints;
4. returns the resulting type.

For example, **application** `DEapp(f, a)`:

```
st1 = tinfer(f)           # T_f
st2 = tinfer(a)           # T_a
st3 = styp_new()          # fresh result type  $\alpha$ 
unify(st1, styp_fun(st2, st3)) # T_f must be T_a  $\rightarrow \alpha$ 
return st3
```

Recursive functions (`DEfix`) are handled by binding the function’s name to a fresh function type *before* checking the body, so the body can reference itself; the body’s inferred result type is then unified with the codomain.

3.3 The type context

`tctx` is a simple linked list of (name, `styp`) pairs. `tctx_search` walks it linearly. This is fine for the assignment’s scale; for a real compiler one would use a hash map.

4. JS code emitter (Item 2d)

The JS emitter is a **direct, structural translation** of LAMBDA terms to JS expressions. The closure-conversion machinery (Section 5) is layered on top of this; the textbook (non-converted) version that uses inner functions is also kept in `solution.py` as `dexp_trx2js_naive` for comparison.

4.1 Translation of each form

LAMBDA term	JS expression (closure-converted)
<code>DEint(n)</code>	<code>n</code>
<code>DEbtf(b)</code>	<code>true / false</code>
<code>DEstr(s)</code>	<code>"..."</code>
<code>DEvar(x)</code>	<code>x or __env__.x</code> if <code>x</code> is captured
<code>DElam(x, e) / DElam1</code>	<code>__mkclos(__fnN, { fv1: ... })</code> (lifted)
<code>DEapp(f, a)</code>	<code>__apply(<f>, <a>)</code>
<code>DEfix(f, x, e) / DEfix1</code>	<code>__mkfix(__fnN, { ... }, "f")</code>
<code>DElet(x, d, s)</code>	desugared to $(\lambda x. s) d$ and emitted as such
<code>DEif0(t, c, a)</code>	<code>(<t> ? <c> : <a>)</code>
<code>DEtup(e1, e2)</code>	<code>[<e1>, <e2>]</code>
<code>DEfst(e) / DESnd(e)</code>	<code>(<e>)[0] / (<e>)[1]</code>
<code>DElist_nil()</code>	<code>[]</code>
<code>DElist_cons(h, t)</code>	<code>[<h>].concat(<t>)</code>
<code>DElazy(e)</code>	<code>__mklazy(__lzN, { ... })</code> (lifted)
<code>DEstcn_nil() / DEstcn_cons</code>	<code>null / {tag:'cons', head, tail}</code>
<code>DEarray_size\$val(n, v)</code>	<code>Array(<n>).fill(<v>)</code>
<code>DEarray_size\$fun(n, f)</code>	<code>__arr_from(<n>, <f>)</code> (top-level helper)
<code>DEanno(e, T)</code>	<code><e></code> (type ascriptions are erased at run time)

4.2 Run-time helpers

A small set of helper functions, all declared at the top of every generated program, do the actual closure operations:

```
function __mkclos(code, env) { return { code, env }; }
function __mkfix (code, env, selfname) {
  var c = { code, env };
  c.env[selfname] = c;
  return c;
}
function __apply (c, x) { return c.code(c.env, x); }
function __mklazy(code, env) {
  return { code, env, memo: false, value: null };
}
function __force (t) {
  if (!t.memo) { t.value = t.code(t.env); t.memo = true; }
  return t.value;
}
function __arr_from(n, c) {
  var r = [];
  for (var i = 0; i < n; i++) r.push(__apply(c, i));
  return r;
}
function __arr_set (a, i, v) { a[i] = v; return a; }
```

These **seven** helpers, plus the lifted user functions, are the *only* function declarations in any program our compiler emits. Crucially, none of them is *anonymous* and none of them is *nested*.

5. Closure conversion (Item 2e)

5.1 The problem

A naive translation of $\lambda x. \lambda y. x + y$ into JS produces

```
((x) => ((y) => x + y))
```

— there is an inner arrow function $(y) \Rightarrow x + y$ which depends on the **captured** variable x from the surrounding lambda. Languages that don't support nested functions (C, assembly, our restricted JS target) cannot represent this directly. Closure conversion rewrites the program so that every function is at the top level; captured variables are passed in explicitly via an *environment*.

5.2 The transformation

For each $\lambda x. \text{body}$:

1. Compute $FV = \text{freevars}(\text{body}) - \{x\}$.
2. Generate a fresh top-level function

```
function __fnN(__env__, x) { return <body'>; }
```

where body' is body with every free variable $v \in FV$ rewritten to $__\text{env}__.v$.

3. Replace the original lambda with the **closure object**

```
__mkclos(__fnN, { v1: <v1>, v2: <v2>, ... })
```

capturing the current values of the free variables.

For each application $f(a)$:

```
__apply(<f>, <a>)           // which is just  f.code(f.env, a)
```

For each recursive function $\text{fix } f(x). \text{ body}$:

- the function’s own name f is treated as an additional captured variable;
- `__mkfix` is used instead of `__mkclos`; it builds the closure object and then writes `c.env[<selfname>] = c` so that `__env__.<selfname>` resolves correctly inside the body — i.e. the closure can call itself.

For `DElet(x, d, s)`: we **desugar** to $(\lambda x. s) d$ and let the lambda machinery do the work. This guarantees that any captures introduced by let-bindings are handled identically to lambda captures.

5.3 Implementation in `solution.py`

The conversion is a single recursive function:

```
cc(de, env_vars, state)
```

- `env_vars` is the set of variables that, in the **currently emitted JS function**, must be read via `__env__`. Variables outside this set are parameters or local lets and are emitted as bare identifiers.
- `state.lifted` accumulates the top-level function declarations generated so far.
- `state.fresh()` produces fresh names `__fn0`, `__fn1`, ...

When `cc` enters a `DElam`, it (a) computes the free variables, (b) creates a fresh `fname`, (c) recursively closes-over the body **with `env_vars` set to the lambda’s free-variable set**, (d) appends the top-level declaration to `state.lifted`, and (e) returns the `__mkclos(...)` expression that captures the current values at the call site.

The trick that makes nested cases (e.g. three-deep $\lambda x. \lambda y. \lambda z. x + y + z$) fall out automatically: when emitting the *enclosing* lambda’s environment object, each free variable v is itself emitted as either `v`: `v` (if v is a parameter or local in the enclosing function) or `v`: `__env__.v` (if v is *also* free in the enclosing function and therefore captured one level up).

5.4 Worked example

Source:

```
λx. λy. λz. (x + y) + z
```

Generated JS:

```
function __fn2(__env__, z) { return ((__env__.x + __env__.y) + z); }
function __fn1(__env__, y) { return __mkclos(__fn2, { x: __env__.x, y: y }); }
function __fn0(__env__, x) { return __mkclos(__fn1, { x: x }); }
__mkclos(__fn0, {})
```

Note especially `{ x: __env__.x, y: y }` in `__fn1`: `x` is being *passed through* (it was captured one level up; we read it from this function’s own environment), while `y` is being *captured fresh* (it is the parameter of `__fn1` and is now closed over for `__fn2`).

5.5 Verification

The 15-test suite in `TEST/run_tests.py` exercises every relevant language feature: nested lambdas, recursion (`fix/fix1`), `let`, tuples, lists, lazy values, streams, and arrays. Every test:

- type-checks correctly;
- emits JS containing **zero** arrow functions and **zero** nested function expressions;
- produces the expected numerical answer when run under Node.js.

6. Summary of files

```
07/MySolution/  
|-- solution.py      -- the compiler (type-inference + emitter + CC)  
|-- REPORT.pdf      -- this report  
'-- TEST/  
    |-- run_tests.py -- 15-test driver (Python + Node)  
    '-- README.md    -- description of the test suite
```

Run all tests with:

```
python3 TEST/run_tests.py
```